

Comparing Nearest Neighbor, Clarke-Wright Savings, and Harmony Search for the Capacitated Vehicle Routing Problem

Mandy Jones, Tomu Yamashita, Tristen Martin, Charles Davis

Abstract

The Capacitated Vehicle Routing Problem (CVRP) is an optimization problem where the objective is to determine efficient routes that can satisfy customer demand without exceeding vehicle capacity constraints and ensure that each customer is visited exactly once. This study compares three algorithms to solving this problem: Nearest Neighbor, Clarke-Wright Savings, and Harmony Search. The performance of these algorithms is analyzed based on percentage gap from the best-known CVRPLIB solution, computational efficiency, and their ability to generate feasible routes for CVRP instances. Each algorithm is tested ten times on problem sizes ranging from 30 to 200 customers. The most notable finding was that Nearest Neighbor has the fastest runtimes with the least efficient routes; Clarke-Wright Savings had moderate speed with good routes; and Harmony Search generated the most efficient routes with the slowest run time.

1. Problem Definition

The Vehicle Routing Problem (VRP) seeks the optimal set of routes for a fleet of vehicles delivering goods to customers while minimizing total travel distance. This study focuses on the Capacitated Vehicle Routing Problem (CVRP), where each vehicle has a fixed carrying capacity that limits how much demand it can serve per route, making the problem harder to solve. The CVRP is NP-hard, since it reduces to the Traveling Salesman Problem when using a single, unlimited-capacity vehicle, meaning no known algorithm can guarantee an optimal solution in polynomial time, which is why heuristic approaches are used instead.

2. Algorithm Explanations

2.1 Nearest Neighbor Heuristic

In this comparison study, the Nearest Neighbor algorithm serves as the baseline. This is because it is the simplest approach: it performs the closest thing it hasn't yet. To explain, it starts at the depot, which is the starting and ending point for the truck's route. It then goes to the unvisited customer that is the closest and fits into the truck's remaining space. The algorithm repeats this until no more customers fit, and then it sends the truck home and starts a new one. It does this until every customer is visited.

2.2 Clarke-Wright Savings Algorithm

The Clarke-Wright Savings algorithm starts by giving every customer their own truck, having the truck go from the depot to that customer and back. It then looks at every possible pair of customers and calculates the savings, which is how much distance would be saved by combining their two separate trips into one that visits both. These savings values are sorted from largest to smallest, with the algorithm merging routes together starting with the largest savings. A merge only happens if the two customers are the first or last customer in the list of their routes and if combining them doesn't exceed the truck's capacity. This process continues until no more valid merges are left.

2.3 Harmony Search

The Harmony Search algorithm starts by creating a harmony memory, which stores several possible customer orderings along with the total distance of the truck routes they produce. It begins with multiple different solutions so that the search can explore several possibilities instead of starting from a single route. Unlike the traditional Harmony Search algorithm, this implementation uses deterministic rules rather than random selection to perform memory consideration and pitch adjustment, ensuring the same input always produces the same output. It then creates new customer orderings by building a sequence one customer at a time, either using information from existing solutions in memory (memory consideration) or selecting a different available customer. The algorithm can also adjust choices by moving toward nearby customers (pitch adjustment) to improve the route. Each new ordering is converted into feasible truck routes and refined by swapping neighboring customers when those swaps reduce the total distance. If the improved solution is better than the worst solution currently stored in memory, it replaces that solution. This process repeats a set number of iterations, and the best solution remaining in the harmony memory is returned.

3. Theoretical Complexity

3.1 Nearest Neighbor Heuristic

The Nearest Neighbor algorithm has a time complexity of $\Theta(n^2)$. Every time a customer is added to a route, the algorithm must scan all unvisited customers to identify the next closest one.

3.2 Clarke-Wright Savings Algorithm

Clarke-Wright Savings algorithm has a time complexity of $\Theta(n^2 \log n)$. Calculating the savings value for every pair of customers requires comparing all possible pairs, which takes $\Theta(n^2)$ time. The n^2 savings values must then be sorted from largest to smallest so that the most beneficial merges are attempted first. This sorting step takes $\Theta(n^2 \log n)$ time and is the most expensive.

3.3 Harmony Search

Harmony Search has a time complexity of $\Theta(I n^3)$, where I is the number of iterations and n is the number of customers. During each iteration, the algorithm generates and evaluates a new customer ordering before improving it using local search. Constructing each ordering requires repeatedly searching through and removing customers from lists, while the local search evaluates many neighboring orderings to find improvements. Together, these operations require $\Theta(n^3)$ time per iteration. Repeating this process for I iterations gives an overall time complexity of $\Theta(I n^3)$.

4. Experimental Design

This experimental design was structured around four research questions:

1. How does runtime change as the number of customers increases?
 - Runtime goes up for all three algorithms as the number of customers grows, but by very different amounts. Nearest Neighbor grew about 31 times slower, Clarke-Wright grew about 111 times slower, and Harmony Search grew the most dramatically at about 317 times slower. This matches the expectation, since Harmony Search has the most demanding time complexity ($\Theta(n^3)$).
2. Which algorithm provides the best balance between percentage gap from the best-known CVRPLIB solution and execution time?
 - Clarke-Wright offers the best balance overall. It runs much faster than Harmony Search while still keeping its gap from the best-known solution fairly low (7%–12%), whereas Nearest Neighbor is fastest but has a much worse gap (36%–46%), and Harmony Search has the worst gap of all (119%–158%) despite taking by far the longest to run.
3. Does each algorithm's percentage gap from the best-known solution stay consistent as problem size increases, or does it get better or worse?
 - Nearest Neighbor's gap stayed relatively flat (36%–46%) as problem size increased, meaning its performance didn't get noticeably better or worse with scale. Clarke-Wright showed the same pattern, staying consistently low (7%–12%) regardless of problem size. Harmony Search also stayed consistent, but at a much higher and less desirable range (119%–158%), suggesting its poor performance isn't tied to problem size. It's more than likely tied to parameter tuning or implementation details that is driving that gap rather than the number of customers involved.

5. Results & Analysis

Nearest Neighbor Results:

data file	num_customers	num_routes	total_distance	best value	gap_percent	avg_runtime_ms	trials
data/A-n32-k5.vrp	31	5	1146.4	784	46.22	0.5312	10
data/E-n51-k5.vrp	50	5	711.5	521	36.56	1.1336	10
data/E-n101-k8.vrp	100	8	1174.36	815	44.09	3.3567	10
data/M-n200-k16.vrp	199	17	2034.4	N/A	N/A	16.4016	10

As the problem grew from 31 customers to 199 customers, the runtime jumped from 0.53ms to 16.40ms, which is 31 times slower. That makes sense regarding the $\Theta(n^2)$ complexity, since the algorithm must compare every remaining customer at each step. The bigger the problem, the more comparisons that have to be made. However, the quality of the routes stayed in the same range (36% - 46%) as the problem grew, which makes sense because the algorithm doesn't reconsider earlier choices. Its greedy decision-making causes the same degree of inefficiency regardless of problem size.

Clarke-Wright Savings Results:

clarke_wright	data/A-n32-k5.vrp	31	5	843.69	784	7.61	1.1698	10
clarke_wright	data/E-n51-k5.vrp	50	6	584.64	521	12.21	3.1485	10
clarke_wright	data/E-n101-k8.vrp	100	8	889.0	815	9.08	16.0064	10
clarke_wright	data/M-n200-k16.vrp	199	17	1395.74	N/A	N/A	130.4101	10

As the problem grew from 31 to 199 customers, the runtime jumped from 1.17ms to 130.41ms, which is 111 times slower. This is much steeper than shown in the Nearest Neighbor's growth on the same scale. This makes sense as Clarke-Wright Savings' time complexity is $\Theta(n^2 \log n)$; it grows a bit faster because of the extra step of sorting the saving values. Meanwhile, the gap from the best value stayed in a lower, less expansive range from 7% - 12%, since the algorithm considers all possible customer pairings upfront rather than greedily picking what's closest.

Harmony Search Results:

harmony_search	data/A-n32-k5.vrp	31	5	1714.2	784	118.65	155.3155	10
harmony_search	data/E-n51-k5.vrp	50	6	1281.13	521	145.9	416.1114	10
harmony_search	data/E-n101-k8.vrp	100	8	2106.33	815	158.45	2204.2205	10
harmony_search	data/M-n200-k16.vrp	199	17	4915.27	N/A	N/A	49263.7119	10

As the problem grew from 31 to 199 customers, the runtime jumped from 155.32ms to 49263.71ms, which is 317 times slower. This is the steepest growth yet, compared to both Nearest Neighbor as well as Clarke-Wright. Since Harmony Search's time complexity is $\Theta(n^3)$, it

makes sense that the average runtime is much larger. The gap from the best value was consistently more than double, ranging between 119% - 158%. This large gap suggests that the additional time was spent searching for a better path.

6. References

[K-Nearest Neighbor\(KNN\) Algorithm - GeeksforGeeks](#)

[Solving CVRP with Greedy Algorithms](#)

<https://github.com/ngchunlong279645/Capacitated-Vehicle-Routing-Problem-CVRP->

<https://github.com/zuzhaoye/vehicle-routing-problem-vrp-Clarke-Wright-Savings-Method>

[Improved Harmony Search Algorithm](#)

[Critical Analysis on Harmony Search Algorithm](#)

https://github.com/sagniklp/Harmony-Search?utm_source=chatgpt.com